



Partie 6 : Découvrir les notions de base de la programmation

Chapitre 1 : Rôle des algorithmes dans l'informatique

1- Introduction

Les algorithmes sont omniprésents et dirigent notre économie, notre société et peut-être même la façon dont nous pensons. **Mais en fait, qu'est-ce qu'un algorithme ?**

Dans le domaine des mathématiques, dont le terme est originaire, un algorithme peut être considéré comme **un ensemble d'opérations ordonné et fini devant être suivi dans l'ordre pour résoudre un problème.**

Pour effectuer une tâche, quelle qu'elle soit, un ordinateur a besoin d'un programme informatique. Or, pour fonctionner, un programme informatique doit **indiquer à l'ordinateur ce qu'il doit faire** avec précision, étape par étape.

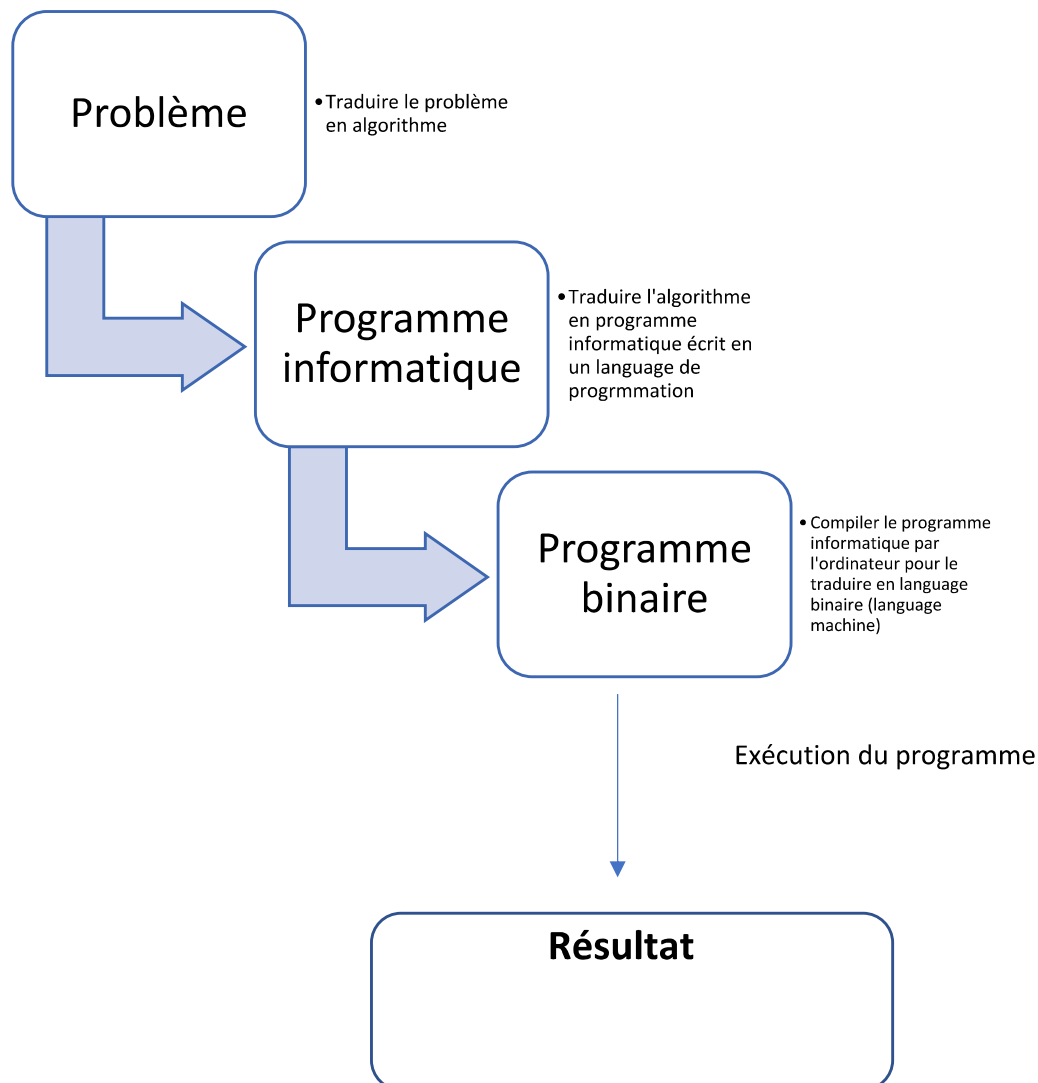
L'**ordinateur " exécute " ensuite le programme**, en suivant chaque étape de façon mécanique pour atteindre l'objectif. Or, il faut aussi dire à l'ordinateur " comment " faire ce qu'il doit faire. C'est le rôle de l'algorithme informatique.

Les algorithmes informatiques fonctionnent par le biais **d'entrées (input) et de sortie (output)**. Ils reçoivent l'input, et appliquent chaque étape de l'algorithme à cette information pour générer un output.



Par exemple, un moteur de recherche est un **algorithme** recevant une **requête de recherche** en guise d'input. Il mène une recherche dans sa base de données pour des éléments correspondant aux mots de la requête, et produit ensuite les **résultats**.

Les ordinateurs ne comprennent pas le langage humain, et un algorithme informatique doit donc être **traduit en code** écrit dans un langage de programmation. Il existe de nombreux langages tels que Java, Python, C, C++... chacun présente des spécificités et convient davantage à un cas d'usage spécifique.



Il faut savoir que le **programme binaire** est constitué d'un ensemble de valeurs constituées de 0 et 1 c'est ce qu'on appelle les **bits**.

Du coup un bit est l'élément de base avec lequel travaille l'ordinateur (nous pouvons le comparer avec vrai/faux, on/off...).

Et voici quelques exemples de valeurs décimales converties en binaires.

Valeur décimale	Valeur binaire
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010



Nous entendons souvent parler d'un octet, alors qu'est-ce qu'est un octet ?

Un octet est un ensemble de 8 bits, en fait un ordinateur ne calcule jamais sur 1 bit à la fois mais sur un ou plusieurs octets.

Pour simplifier **1Byte = 1 octet= 8 Bits**.



Quel est la liaison entre les bits et le code ASCII ?

La mémoire de l'ordinateur conserve toutes les données sous forme numérique. Il n'existe pas de méthode pour stocker directement les caractères. Chaque caractère possède donc son équivalent en code numérique : c'est le **code ASCII** (American Standard Code for Information Interchange).

La représentation des caractères se fait comme suit :

Les caractères de numéro 0 à 31 et le 127 ne sont pas affichables, ils correspondent à **des commandes de contrôle de terminal informatique**. Le caractère numéro 127 est **la commande pour effacer**. Le caractère numéro 32 est **l'espace**. Le caractère 7 provoque **l'émission d'un signal sonore**. Les autres caractères sont les chiffres **arabes**, les lettres **latines majuscules et minuscules sans accent**, des symboles de **punctuation**, des **opérateurs mathématiques** et quelques **autres symboles**.

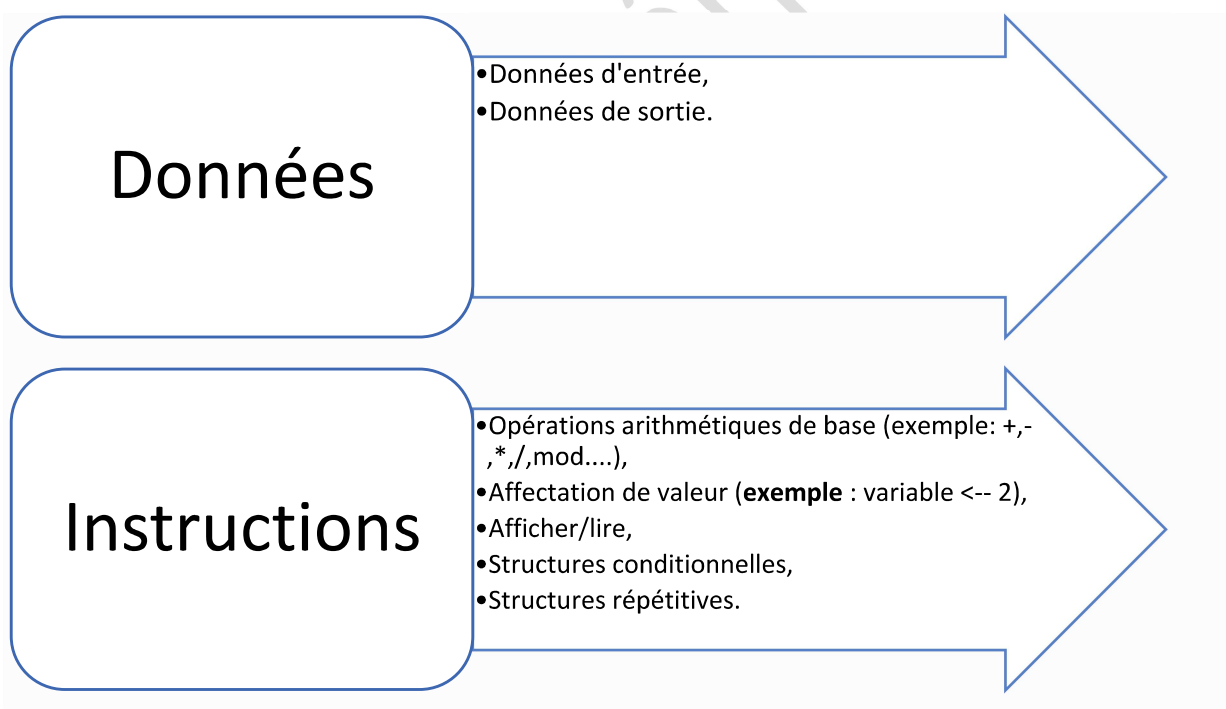


Principe : L'ASCII définit **128** caractères numérotés de 0 à 127 et codés en **binaire** de 0000000 à 1111111. Sept bits suffisent donc. Toutefois, les ordinateurs travaillant presque tous sur un multiple de huit bits (un octet), chaque caractère d'un texte en ASCII est souvent stocké dans un octet dont le 8^e bit est 0.

2- Construction d'un algorithme

Pour chaque problème, il vous est demandé de définir clairement :

- **Les données d'entrée** du problème en précisant leurs types et leur rôle,
- **Les données de sortie** du problème en précisant leurs types,
- **Les différentes instructions** permettant d'obtenir les données de sorties à partir des données d'entrée.



Les données et leur nature



Les **données (d'entrée et de sortie)** manipulées par un **algorithme** peuvent être de nature différente (entiers, réels, caractères, chaînes de caractères, booléens...). En **programmation**, on parle de type plutôt que de la nature de la donnée.

Le **type d'une donnée** spécifie la taille occupée par la donnée en mémoire, les opérations qui lui sont applicables ainsi que l'intervalle de valeurs autorisées.

Il faut savoir que Les **types de bases, types primitifs, types élémentaires** ou encore **types simples**, sont :

- **Entier** : représente l'ensemble des entiers relatifs
- **Réel** : l'ensemble des réels
- **Booléen** : le domaine des booléens (vrai / faux)
- **Caractère** : le domaine des caractères alphanumériques
- **Chaîne** : le domaine des textes

Les primitives d'entrée et de sortie



Les **primitives d'entrée/sortie** permettent un échange d'information entre l'algorithme et l'utilisateur.

```
afficher(expression) // permet d'afficher sur l'écran  
saisir(nomVariable) // lecture par clavier
```

3- La structure d'un algorithme

La structure d'un algorithme est la suivante :

```
Algorithme nomAlgorithme  
    // déclaration des variables  
Début  
    // liste des instructions  
Fin
```

Un algorithme est constitué par un en-tête et un corps, ce dernier étant constitué de deux parties.

- L'**en-tête** permet de donner un **nom** à l'algorithme. Ce nom n'a pas de signification particulière mais doit respecter les règles de formation des identifiants. Le fait de nommer les algorithmes permet de les différencier.
- La **première partie du corps**, encadrée par les mots **Algorithme** et **Début**, a un **rôle descriptif**. En particulier, elle décrit les variables et constantes utilisées dans l'algorithme.
- La **seconde partie du corps**, encadrée par les mots **Début** et **Fin**, a un **rôle constructif**. Elle décrit les traitements à réaliser pour aboutir au résultat recherché. Chaque ligne comporte une seule instruction. L'algorithme commence son exécution sur le mot **Début**, se déroule séquentiellement (ligne après ligne, de la première à la dernière, dans cet ordre) et se termine sur le mot **Fin**.

Déclaration d'une variable



Une variable est un élément informatique permettant de mémoriser une information dont l'algorithme aura besoin au cours de son exécution. Elle désigne donc un emplacement mémoire qui permet de stocker une valeur.

Une **variable** est définie par :

- Un **nom (ou identifiant)** unique qui la désigne,
- Un **type (appelé aussi domaine de définition)** unique qui définit de quel « genre » est l'information associée à l'élément informatique,
- Une **valeur attribuée et modifiée** au cours du déroulement de l'algorithme.

Pour la syntaxe de la déclaration d'une variable est la suivante :

```
variable nomVariable : TypeVariable
```

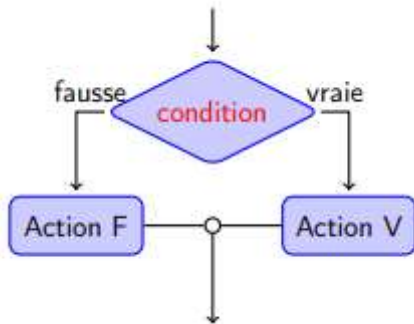
Les instructions élémentaires

Une instruction peut être :

- Un appel à une fonction prédéfinie ou une procédure,
- Utilisation des primitives d'entrée et de sortie,
- Affectation d'une variable,

- Structures conditionnelles,
- Structures répétitives,
- Structures à choix multiple,
- Structures itératives,
- Etc.

Structures conditionnelles



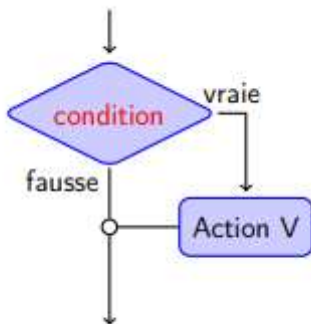
Si condition Alors

 Action Vraie

Sinon

 Action Fausse

Fin Si

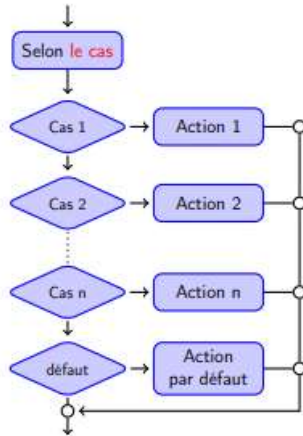


Si condition Alors

 Action Vraie

Fin Si

Structures à choix multiples



Structures répétitives

Selon le cas

Cas 1 : Action 1,

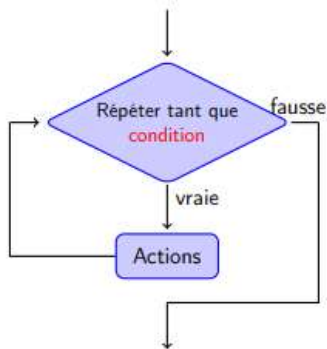
Cas 2 : Action 2,

.....

Cas n : Action n,

Défaut : Action par défaut

Sin Selon



Tant que condition Faire

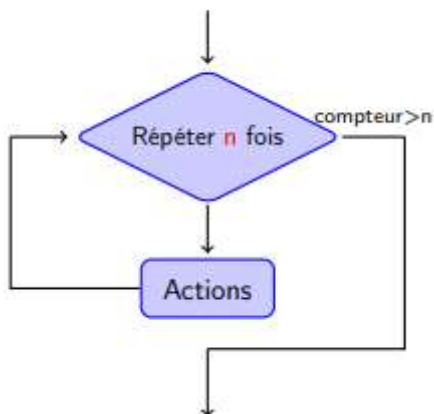
Action 1

Action 2

.....

Action n

Fin Tant que

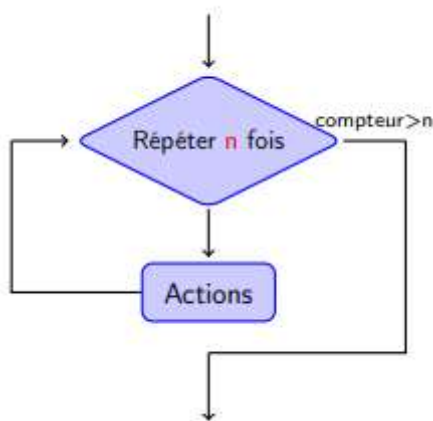


Structures itératives

Répéter

Actions

Tant que condition



Pour i=1 à n Faire

Actions

Fin Pour

Types complexes

Nous avons vu au début les types des données primitives , maintenant nous allons voir les types complexes.

Les types complexes permettent de représenter un ensemble organisé des données, à savoir :

- Les tableaux à une dimension,
- Les tableaux à plusieurs dimensions,
- Les structures des données.

Qu'est ce qu'un tableau ?

Un **tableau unidimensionnel** ou **tableau linéaire** est une variable indicée permettant de stocker plusieurs valeurs de même type. Le nombre maximal d'éléments, qui est précisé à la déclaration, s'appelle la **dimension** (ou capacité) du tableau. Le **type du tableau** est le type de ses éléments. La position d'un élément s'appelle **indice** ou rang de l'élément.

La déclaration d'un tableau à une dimension :

Variable nomVariable : TypeVariable [dimension]